

27 | 重复消费：高并发场景下怎么保证消息不会重复消费？

2023-8-18 邓明



你好，我是大明。今天我们来讨论一个在消息队列里面非常常见的话题——重复消费。

通过前面几节课的学习，我相信你已经发现了，很多方案都会引起一个问题：消息重复发送或者重复消费。而解决思路基本上一致，就是把消费者设计成幂等的。也就是说，同一个消息，不管消费多少次，系统状态都是一样的。

另外一个经常和幂等联系在一起的话题就是重试。就像你在微服务部分接触到的那样，为了提高可用性，我们经常会引入各种重试机制。而重试的一个前提就是重试的动作必须是幂等的。所以，在面试中幂等是一个绕不开的话题。只不过大部分人在讨论幂等的时候，只能想到使用唯一索引，而且即便回答唯一索引，也很难深入。

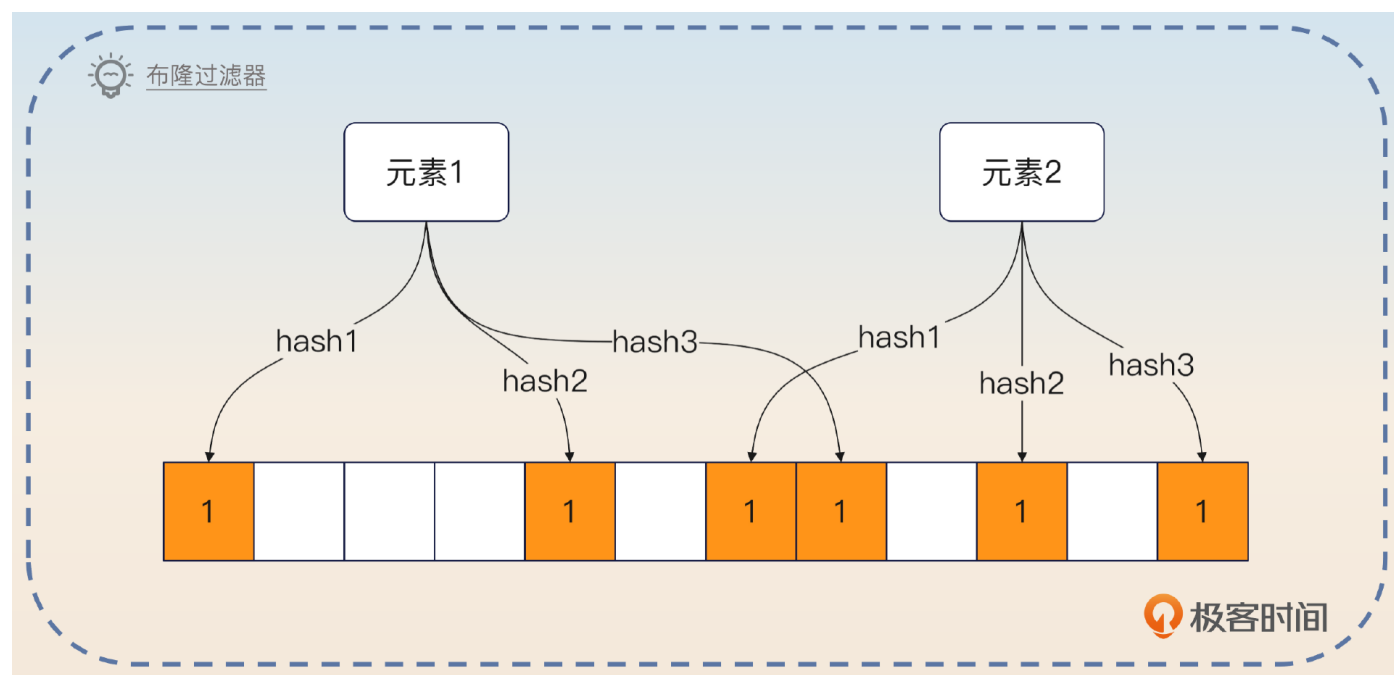
所以今天我就带你从重复消费讨论到幂等，最后给出一个非常强大的高并发幂等方案，确保你在面试的时候可以赢得竞争优势。

布隆过滤器

布隆过滤器（Bloom Filter）是一种数据结构，它可以用于检索一个元素是否在一个集合里。它的优点是空间效率和查询时间都远远超过一般的算法，缺点是存在假阳性的问题，还有对

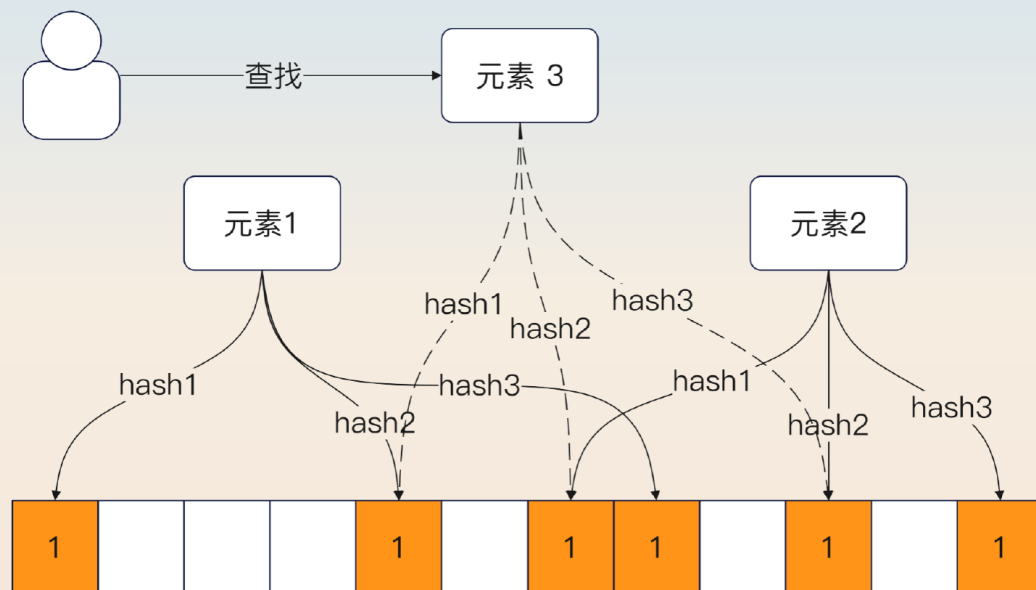
于删除操作不是很友好。

布隆过滤器的基本思路是当集合加入某个元素的时候，通过哈希算法把元素映射到比特数组的 N 个点，把对应的比特位设置成 1。



在查找的时候，只需要看对应的比特位是不是 1，就可以粗略判断集合里有没有这个元素。

如果查询的元素对应的 N 个点都是 1，那么这个元素**可能**存在。如果布隆过滤器认为一个元素存在，但是实际上并不存在，也叫做假阳性。



任何一个点是 0，那么这个元素必然不存在。

因此布隆过滤器经常和其他手段结合在一起判断某个元素在不在。它在判断某个元素必定不存在的场景下，效果非常好。

和布隆过滤器类似的还有 bit array，也叫做 bit map。它也是用一个比特位来代表元素是否存在，1 代表存在，0 代表不存在。它和布隆过滤器的核心区别是它不需要哈希函数，因为它的值本身就是一个数字。比如说，你的用户 ID 是数字，那么你就可以把 ID 当成 bit array 的下标，对应位置的比特位是 1。

ids=[3, 4, 7, 9]



并且，bit array 不存在假阳性的说法，它的判断是精确的。

重复消费的可能原因

重复消费的可能原因有2种。

1. 生产者重复发送。比如说你的业务在发送消息的时候，收到了一个超时响应，这个时候你很难确定这个消息是否真的发送出去了，那么你就会考虑重试，重试就可能導致同一个消息发送了多次。
2. 消费者重复消费。比如说你在处理消息完毕之后，准备提交了。这个时候突然宕机了，没有提交。等恢复过来，你会再次消费同一个消息。

在避免重复消费的实践中你就记住一个原则：**一定要把消费逻辑设计成幂等的**。你的微服务也要尽可能设计成幂等的，这样上游就可以利用重试来提高可用性了。另外我要说明一点，现在大多数消息中间件都声称自己实现了恰好一次（exactly once）语义，都是依赖于重试和幂等来达成的。

面试准备

在面试前，你需要弄清楚一些信息。

你负责的业务里面有没有接口或消费者是要求幂等的？如果有，你是如何解决幂等的？

如果你依赖于唯一索引来解决幂等，那么这部分的写流量有多大？

如果你依赖于唯一索引来解决幂等，那么你是怎么保证业务操作和将数据插入到唯一索引是同时成功，或者同时失败的？

你或者你的同事有没有因为没有设计幂等，或者幂等方案有问题而引起线上事故？如果有，你是怎么解决的？

你的业务是否使用过布隆过滤器，如果有，当时用来解决什么问题？

重复消费、重试、幂等是我们设计系统的时候经常要解决的问题，所以你可以在项目介绍或者自我介绍的时候主动提起你设计过比较复杂的幂等解决方案。还有在面试过程中，如果你和面试官聊到了布隆过滤器，那么你可以主动提起你用布隆过滤器实现过幂等方案。类似地，聊到了 bit array 也可以提起你的幂等方案。当你们聊到 Redis 的 key 过期时间该怎么确定的时候，你可以用幂等方案里的 Redis key 过期时间的计算方式来作为例子。

下面我们来看一下关于重复消费的基本面试思路。

基本思路

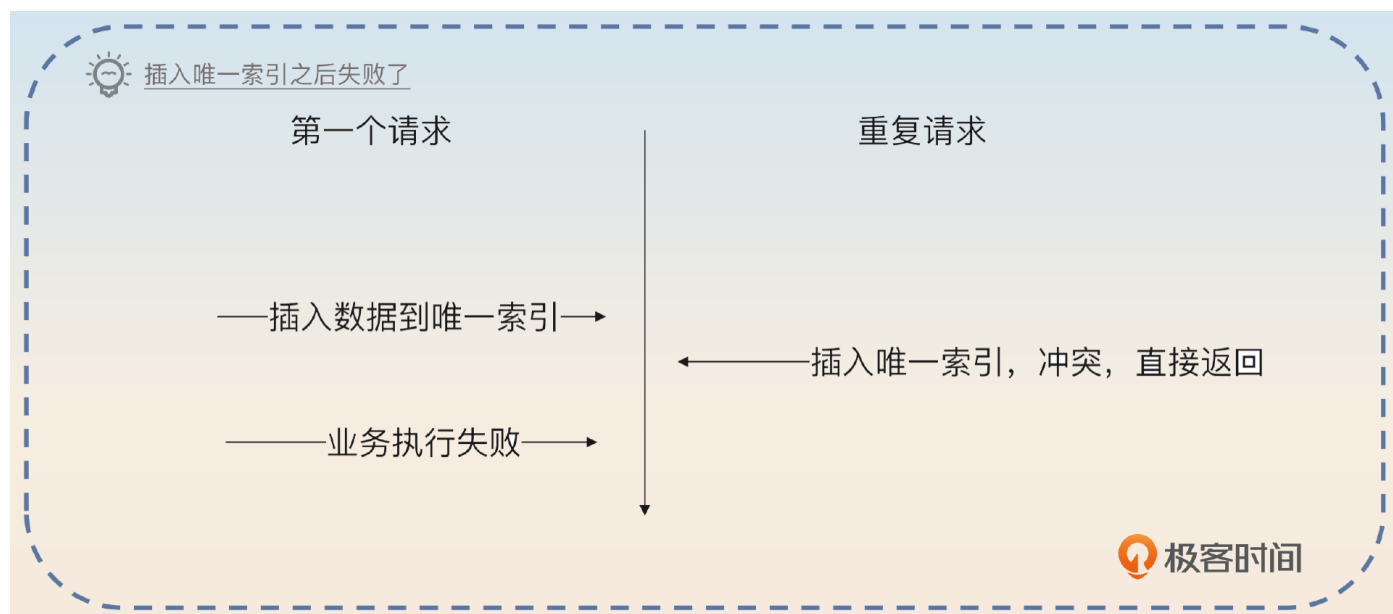
在面试的时候，不管你是提到重试还是提到重复消费，面试官大概率都会问你如何解决幂等的问题，你可以先介绍一个简单的思路，就是**利用唯一索引**。

最简单的幂等方案就是利用唯一索引。比如说在业务处理的时候，先根据消息内容往业务表里面插入一条数据，而这个业务表上有唯一索引。如果插入成功了就说明这个消息没有被处理过，可以继续处理。如果插入的时候出现了唯一索引错误，那就说明这个消息之前被处理过了。

如果想要用这个简单的方案刷出亮点，就需要补充足够多的细节。第一个亮点就是深入讨论究竟是怎么将数据插入到唯一索引的。

本地事务将数据插入到唯一索引

在这个方案里面，当我第一次处理请求的时候，我把数据插入到唯一索引成功了，万一我后面处理业务失败了怎么办？



这个时候你应该使用**事务**。也就是说，你收到消息之后就开启一个本地事务。在这个本地事务里面你会同时完成业务操作和将数据插入到唯一索引这两个操作，然后提交。当然，很多情况下这个插入唯一索引的操作本身就是你业务操作的一部分。

💡 RR 隔离级别下使用本地事务

第一个请求

没有冲突

—— 开启本地事务 ——>

—— 插入数据到唯一索引 ——>

这一步执行完毕就认为业务请求处理完毕

—— 业务操作 ——>

—— 提交本地事务 ——>

重复请求

—— 开启本地事务 ——>

—— 插入数据到唯一索引，阻塞 ——>

因为唯一索引是同一个值，所以你会被阻塞在这里

—— 唯一索引冲突 ——>

在第一个请求提交后，你就会冲突

—— 回滚 ——>

极客时间

在这种机制之下，只会出现一种情况，就是事务提交了，但是提交消息失败了。显然你会再次消费同一条消息，但因为你的事务都已经提交了，所以你下次消费的时候就会遇到唯一索引冲突的错误。

💡 提交消息失败场景

第一个请求

没有冲突

—— 开启本地事务 ——>

—— 插入数据到唯一索引 ——>

这一步执行完毕就认为业务请求处理完毕

—— 业务操作 ——>

—— 提交本地事务 ——>

—— 提交消息失败 ——>

到这一步，实际上你的业务已经执行完毕了

重复请求

—— 开启本地事务 ——>

—— 插入数据到唯一索引，索引冲突 ——>

—— 回滚 ——>

极客时间

如果你没有提交本地事务呢？那就说明你这个消息消费本身就失败了，应该再次重试。所以你可以进一步补充这个细节。

要使用唯一索引，最好的方式是把唯一索引和业务操作组合在一起，做成一个事务。也就是在收到消息的时候先开启事务，把数据插入到唯一索引，然后执行业务操作，最后提交事务。提交事务之后，就认为业务已经成功了，就算接下来提交消息失败了也没关系，因为后面重复请求还是会被唯一索引拦下来。不过万一不能使用本地事务，比如说在分库分表的条件下，那么解决方案就会更加麻烦。

这最后一句可以把话题引向另一个亮点。

非本地事务将数据插入到唯一索引

如果没有本地事务，业务操作和将数据插入到唯一索引的操作就不能看作是一个整体，无法保证要么都成功，要么都失败。这时候，只能追求最终一致性，依赖于一个第三方来检测了。

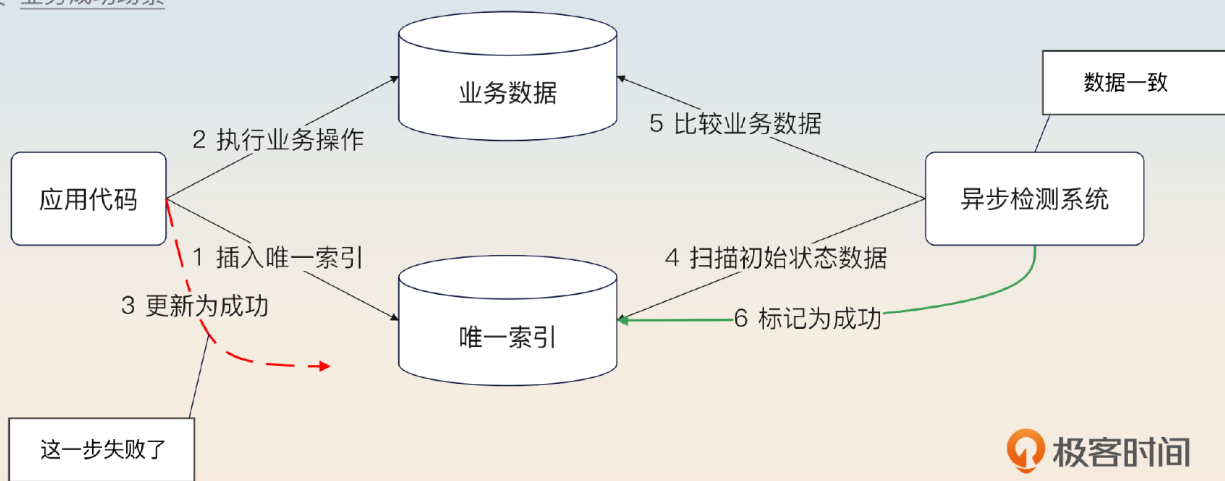
这种情况下整个方案的执行步骤也不一样，它分成3步。

1. 把数据插入到唯一索引，但是这个时候状态被标记为初始状态。注意这一步一定要先执行，这是避免重复处理的关键。
2. 执行业务操作。
3. 将唯一索引对应的数据标记为完成状态。

出问题的地方就是第二步成功了，但是第三步失败了。这时候你就需要使用一个异步检测系统，这个异步检测系统会**定时扫描**唯一索引的表，然后再去扫描业务表。这个时候会有两种情况。

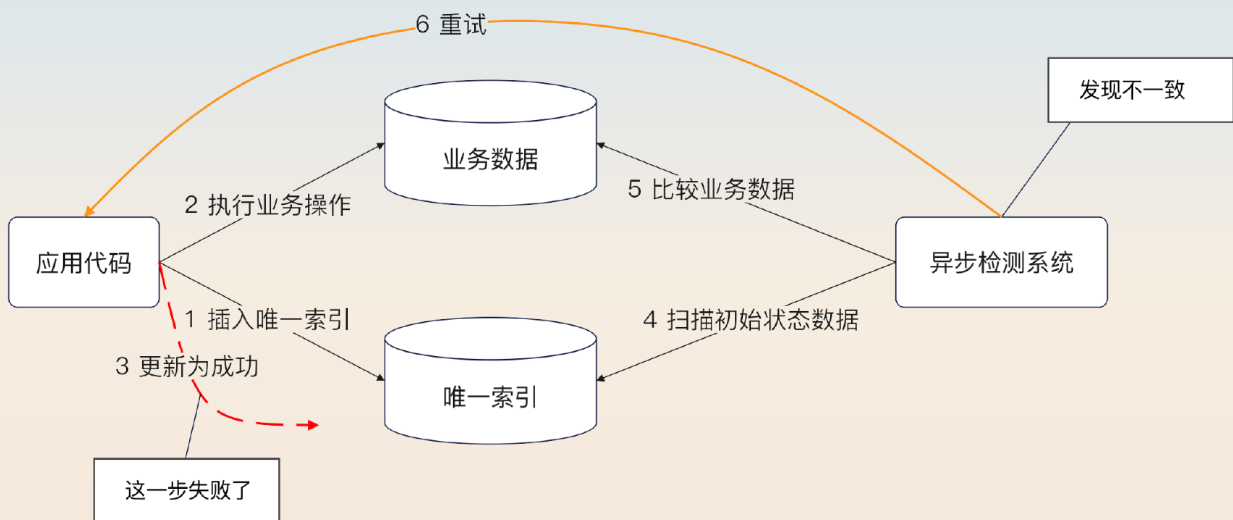
如果业务表的数据表明业务操作已经处理成功了，那么这个异步检测系统就会把唯一索引更新为成功状态。

业务成功场景



如果业务表的数据表明业务操作没有成功，那么异步检测系统可以直接触发重试。

业务失败场景



所以你抓住关键词**异步检测**来回答。

在不能使用本地事务的时候，实现幂等就更加麻烦了，因为我们不能把业务操作和将数据插入到唯一索引这两步做成原子操作。所以我们的解决方案是追求最终一致性，基本步骤是这样的。首先把数据插入到唯一索引里面，避免重复消费，这个时候数据保持在初始化状态。然后执行业务操作，执行业务操作之后，把唯一索引里的数据更新为成功状态。

那么会出问题的地方就是第二步成功了，但是第三步失败了。在这种场景下，需要启动一个异步检测系统定时扫描初始状态的唯一索引数据。这个异步检测系统会检测唯一索引的数据和业务数据，判断是否一致。如果不一致，那么如果这个时候业务操作已经成功，那么就把唯一索引的数据标注为成功；如果这个时候业务失败了，那么就应该触发重试。

到这一步，我觉得就算是单纯用唯一索引这个解决方案来面试，你也可以和其他面试者已经拉开差距了。不过我还有一个更加强大的方案，能让你的优势更加明显。

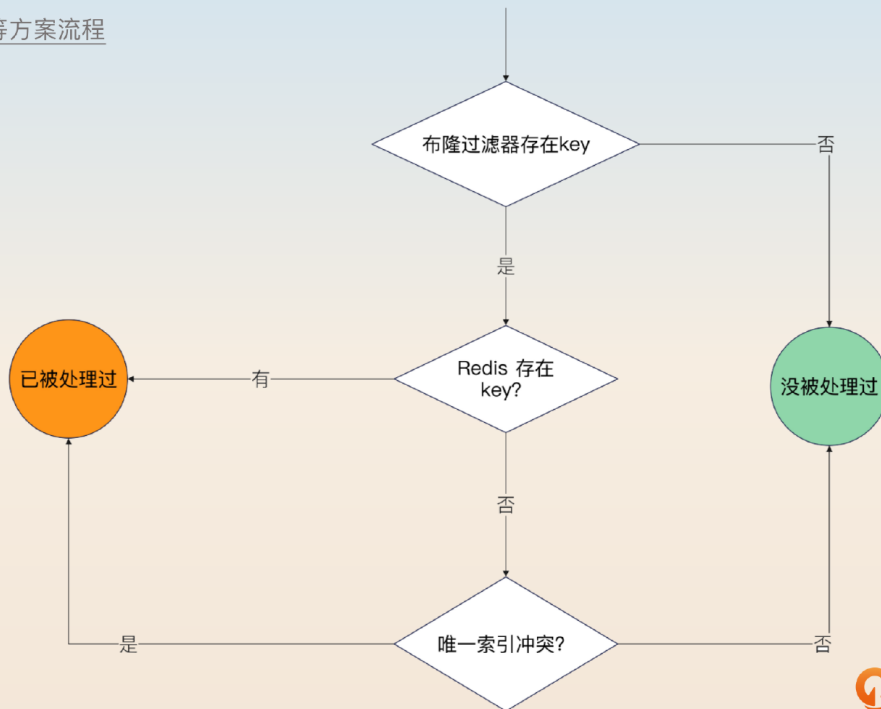
亮点方案：布隆过滤器 + Redis + 唯一索引

这里我给你一个非常高级的方案，这个方案综合运用了大数据处理中常见的布隆过滤器、Redis 和唯一索引。从思路上来说，就是四个字**层层削流**。从目标上来说，就是**确保到达数据库的流量最小化**。

前面的基本方案里我们讨论的是使用唯一索引，并且提及了这种方案的缺陷，就是性能完全取决于数据库的性能。很明显，数据库是撑不住高并发的，尤其是高并发写流量。所以我们就要想尽一切办法让流量在到达数据库之前就返回了。

这时候我们就需要一个高效的数据结构，帮助我们判断某个请求或者消息是否已经被处理过了。布隆过滤器就非常适合用来解决这个问题。但是布隆过滤器本身存在假阳性的问题。也就是说，一个消息明明没有被处理过，但是布隆过滤器可能误报你处理过了。所以我们可以布隆过滤器之后再加一个 Redis，存储近期处理过的业务 key。

所以整个流程就变成了图片中展示的这样。



在面试的时候你可以先介绍下这个方案的基本流程。

我在公司设计过一个高并发的幂等方案。这个幂等方案需要用到布隆过滤器、Redis 和唯一索引。方案的基本思路是，如果依赖于数据库唯一索引来判断幂等，那么数据库撑不住高并发。所以我们就想办法在使用唯一索引之前，尽可能先削减流量。这个场景就非常适合使用布隆过滤器。而为了避免假阳性的问题，进一步降低发送到数据库的流量，在布隆过滤器和数据库之间再引入一个 Redis。

基本流程是这样的：

首先，一个请求过来的时候，我们会利用布隆过滤器来判断它有没有被处理过。如果布隆过滤器说没有处理过，那么就确实没有被处理过，可以直接处理。如果布隆过滤器说处理过（可能是假阳性），那么就要执行下一步。

第二步就是利用 Redis 存储近期处理过的 key。如果 Redis 里面有这个 key，说明它的确被处理过了，直接返回，否则进入第三步。这一步的关键就是 key 的过期时间应该是多长。

第三步则是利用唯一索引，如果唯一索引冲突了，那么就代表已经处理过了。

这个唯一索引一般就是业务的唯一索引，并不需要额外创建一个索引。

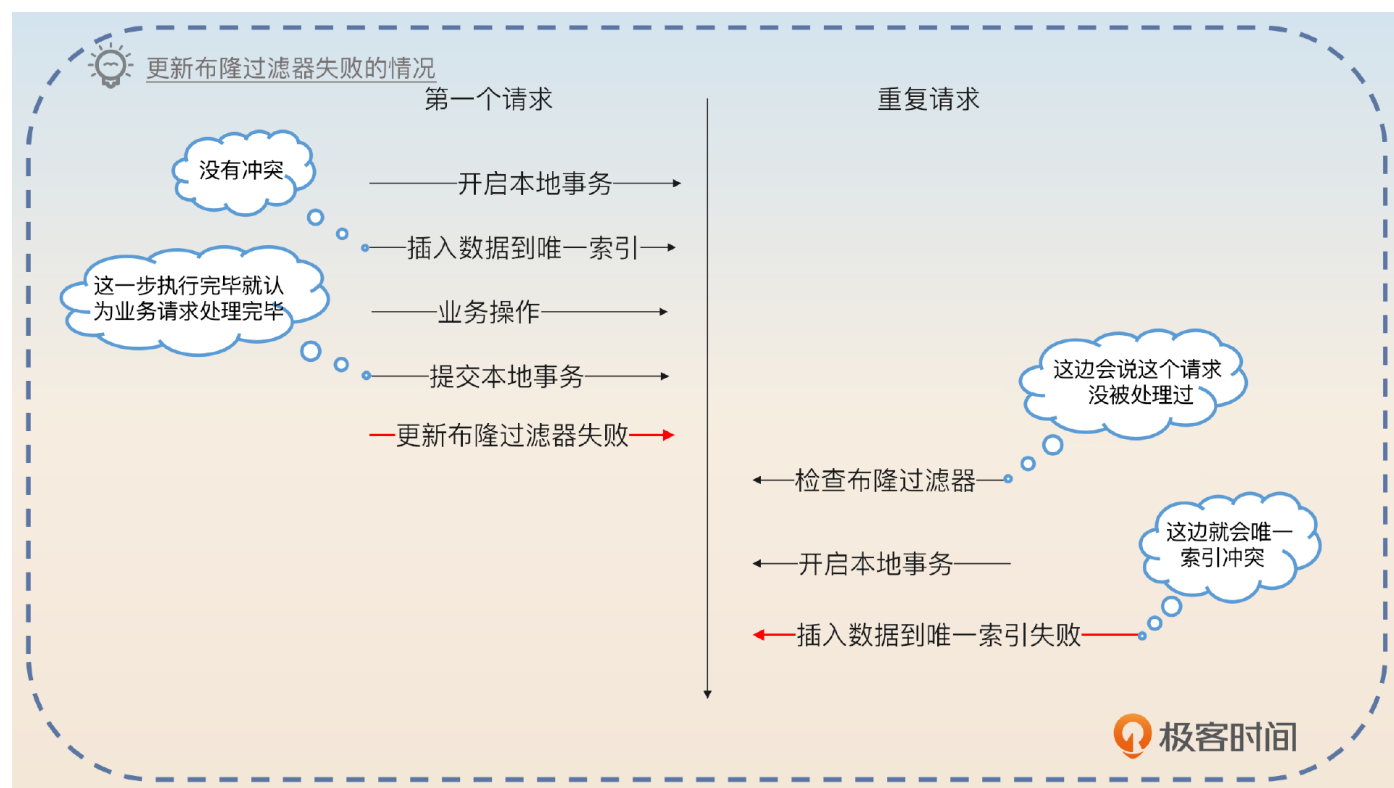
面试官大概率会先问你布隆过滤器的基本知识，然后追问其中的一些细节，现在我们一个一个看。

更新顺序

第一个问题：业务方第一次处理完这个请求，我们该怎么更新这个系统？先更新布隆过滤器，还是先更新 Redis，或者是先更新数据库的唯一索引？

答案是先更新数据库的唯一索引，因为数据库里的数据是最准确的。

如果业务方是第一次执行这个请求，它需要把更新数据库的操作放到自己的业务本地事务里面。等业务方提交的时候，一起提交。在数据库事务提交之后，再去更新布隆过滤器和 Redis。这时候失败了影响也不大，因为最终重复请求被处理的时候，会因为唯一索引冲突而报错，这时候我们就知道这个请求是重复请求了。



后面无论是先更新布隆过滤器，还是先更新 Redis，或者并发更新，都是可以的，这一点不太重要。

可以说这个方案最终都是要依靠唯一索引来兜底的。也就是说不管什么原因导致布隆过滤器或者 Redis 没生效，只要跑到了插入唯一索引这一步，都可以确保你最终不会重复处理消息

或者请求。

Redis key 的过期时间

刚刚我提到了一个关键点，就是 key 的过期时间应该多长这个问题。简单来说，Redis 的作用就是在布隆过滤器之后进一步削减流量，而 key 的过期时间就决定了削减流量的效果，所以只需要确保重复请求过来的时候，这个 key 还没过期就可以了。

如果面试官问起来，你可以这么回答。

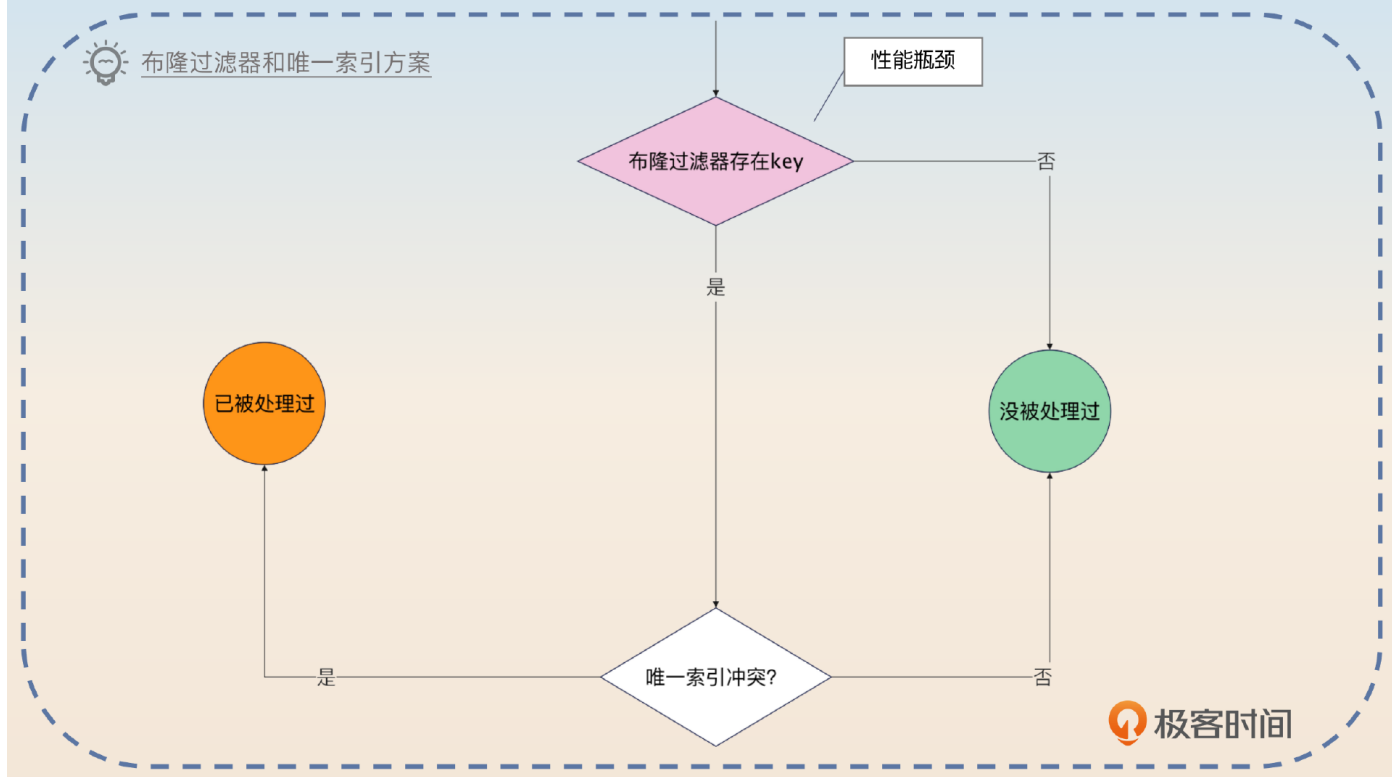
Redis 这一步是为了进一步削减流量，关键就是要确保重复请求过来的时候，key 还没过期。举个例子，假如说预计某个特定业务的重试请求会在 10 分钟内到达，那么可以把过期时间设置成 11 分钟，多出来的一分钟就是余量。

这个地方你也可以进一步总结一下。

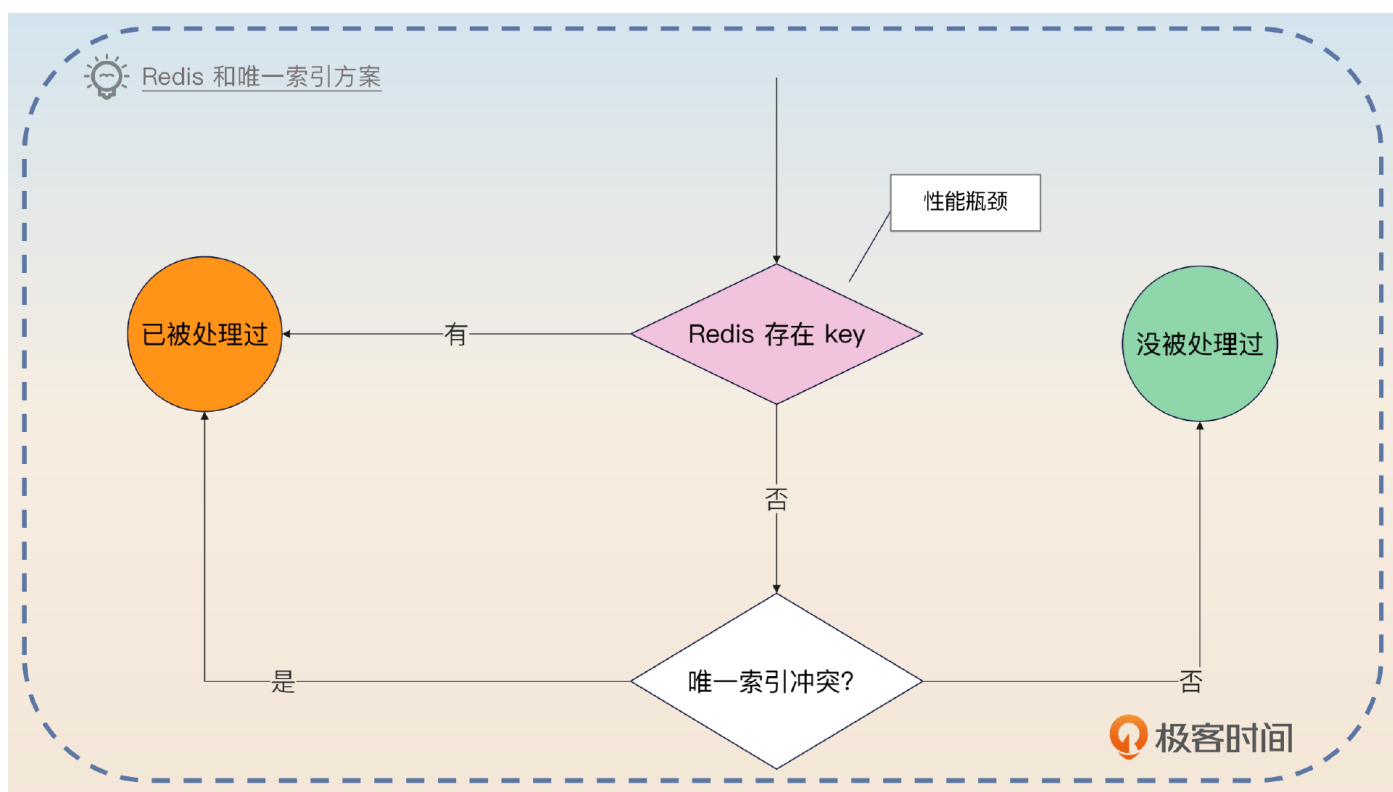
但是如果并发非常高，以至于 key 非常多，也要考虑 Redis 能不能放下这么多 key。另外一个就是有些业务的重试间隔非常长，比如说一小时，这就不太适合引入 Redis 了。可以考虑使用这个方案的简化版本。

简化方案

这个三合一的方案，也可以简化成二合一方案。第一种就是布隆过滤器+唯一索引。这种方案就比较适合重试间隔非常大的业务。



还可以简化成Redis + 唯一索引，如果 Redis 资源足够，而且数据库性能比较差，那么这个方案要更好一点。



你可以介绍一下这两个方案。

如果说业务并发不是特别高，或者说不想用这么复杂的方案，那么可以考虑使用简化方案。

第一种简化方案就是只用布隆过滤器和唯一索引。如果 Redis 资源不足，又或者重复请求间隔太长，导致使用 Redis 的效果不好，那么就比较适合用这个简化方案。第二种简化方案就是只用 Redis 和唯一索引。Redis 资源多，又或者担心布隆过滤器的假阳性问题，就可以采用这个方案。

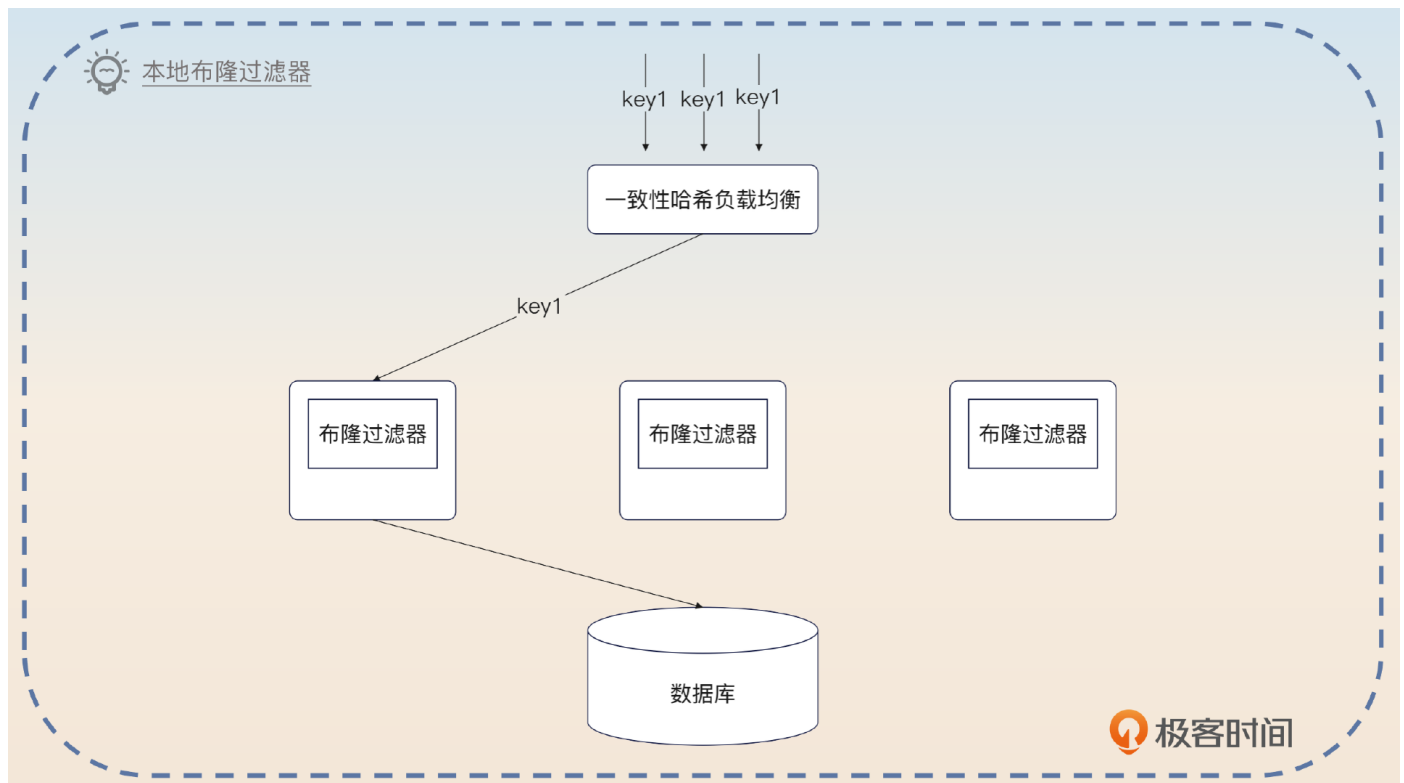
最后总结一下。

其实最开始的时候我是建议用第一个简化方案的，后面如果发现假阳性问题非常严重，那么就可以引入 Redis。

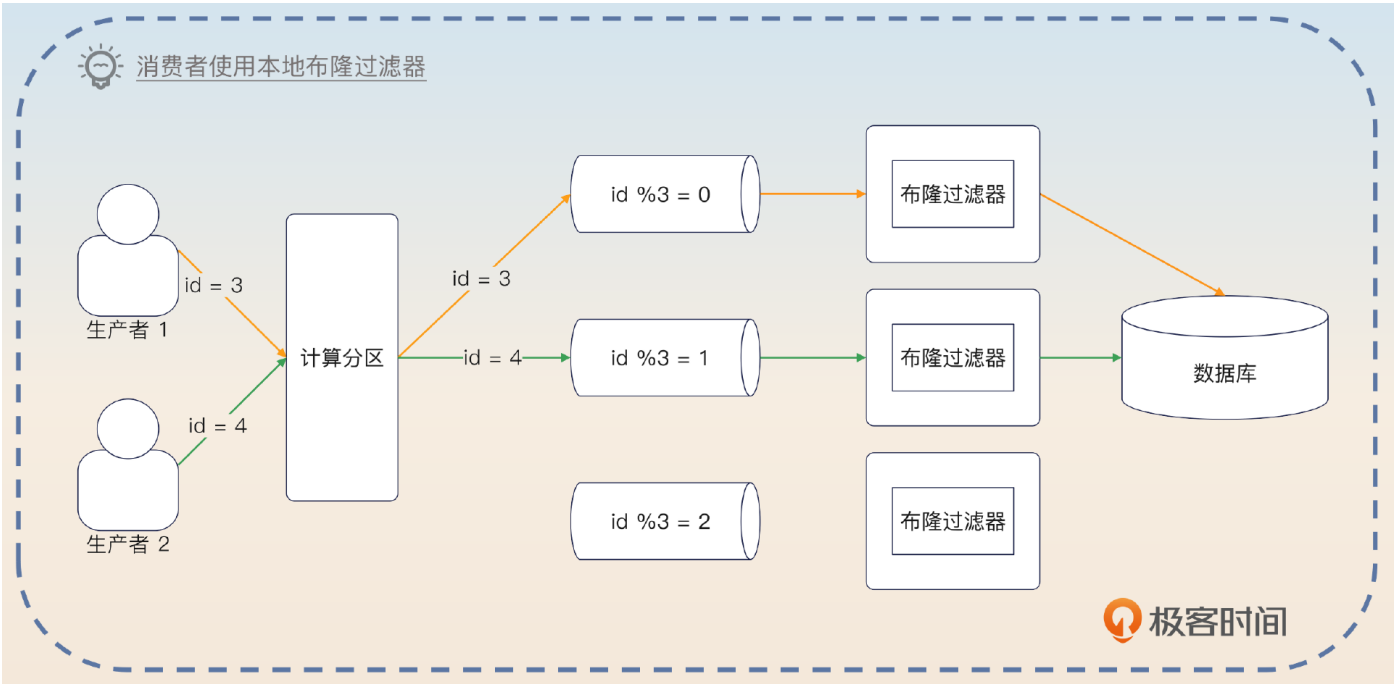
接下来，我们从优化性能的角度来看，第一个可以考虑的就是本地布隆过滤器。

本地布隆过滤器

当看到本地两个字的时候，我不知道你有没有想起前面很多节课里面，我都提过类似的解决方案。这里我要再一次使用本地内存来进一步提高性能了。整体思路是利用一致性哈希等类型的算法执行负载均衡，确保同一个 key 的请求落到同一个实例上，那么我就可以在这台机器上使用基于本地内存的布隆过滤器了。



当然，在消息队列的场景下，这个问题就变成了在发消息的时候要把同一个业务的消息发到同一个分区。这样就可以在消费者身上使用基于本地内存的布隆过滤器了。



而在使用了本地布隆过滤器之后，也可以考虑把 Redis 替换为本地内存。不过一般来说本地内存不多的话，还是使用 Redis 比较好。因为能够通过布隆过滤器的请求就已经是少数了，不值得浪费宝贵的本地内存换取一点点性能的提升。

所以你可以进一步阐述这个方案，抓住关键词**本地布隆过滤器**。

在性能要求非常苛刻的情况下，可以考虑使用本地布隆过滤器，但是这要和负载均衡结合在一起使用。比如说在消息消费的场景下，应该要求生产者把同一个 key 的消息都发到同一个分区上，这样对应的消费者就可以使用本地布隆过滤器了。

紧接着你可以补充一段话，关键词就是**重建本地布隆过滤器**。

本地布隆过滤器在服务器重新启动之后，重建起来也很简单，基本上有两种思路。一种就是不用重建，直接处理新请求。在处理新请求的过程中，逐步重建起布隆过滤器。这种思路适合时效性很强的请求，比如说今天就不可能收到昨天的重复请求这种场景。

另外一种思路就是利用过去一段时间的数据，比如说我预计我今天收到的重复

请求最多来自三天前，那么我就用这三天内处理过的请求来构建布隆过滤器。

布隆过滤器不准确也没关系，反正有唯一索引兜底，只需要小心不要让太多流量最终落到唯一索引上就可以。

使用本地布隆过滤器就已经很快了，但如果你还想进一步提高性能，那么就可以考虑下使用布隆过滤器的替代品。

布隆过滤器的替代品

在一些场景下，你可以用更加高效的数据结构 —— bit array。如果 key 本身就是数字，比如数据库某张表的自增主键，像这种情况下，bit array 性能更好，并且更能节省内存。

那么你面试有两种策略，一种策略就是直接在前面介绍方案的基本流程的时候，你就把布隆过滤器换成 bit array。另外一种策略就是补充说明，强调这个方案**可以自由切换到 bit array 上**。

我这里还在布隆过滤器这边做了一个抽象，也就是说，对于一些业务，其实我这里可以把布隆过滤器换成 bit array 的结构。比如说某个业务要求判断幂等，用的就是业务的自增主键，那么他们就可以使用 bit array 这个实现来判断幂等。这里也有一个小技巧，就是假如说自增主键的起点是 N ，那么在 bit array 的第一位就可以表示为 N ，第二位表示为 $N + 1$ ，这样就能进一步节省内存。

最后的小技巧是平时我使用 bit array 的时候使用的。有些时候为了防止竞争对手猜到自己的业务量，自增主键不是从 0 开始的，又或者在使用分布式 ID 的时候，ID 也不是从 0 开始的。假如说最小的 ID 是 5,000,000。那么 bit array 里的第一个比特位就可以表示为 5,000,000，第二个比特位就表示为 5,000,001。

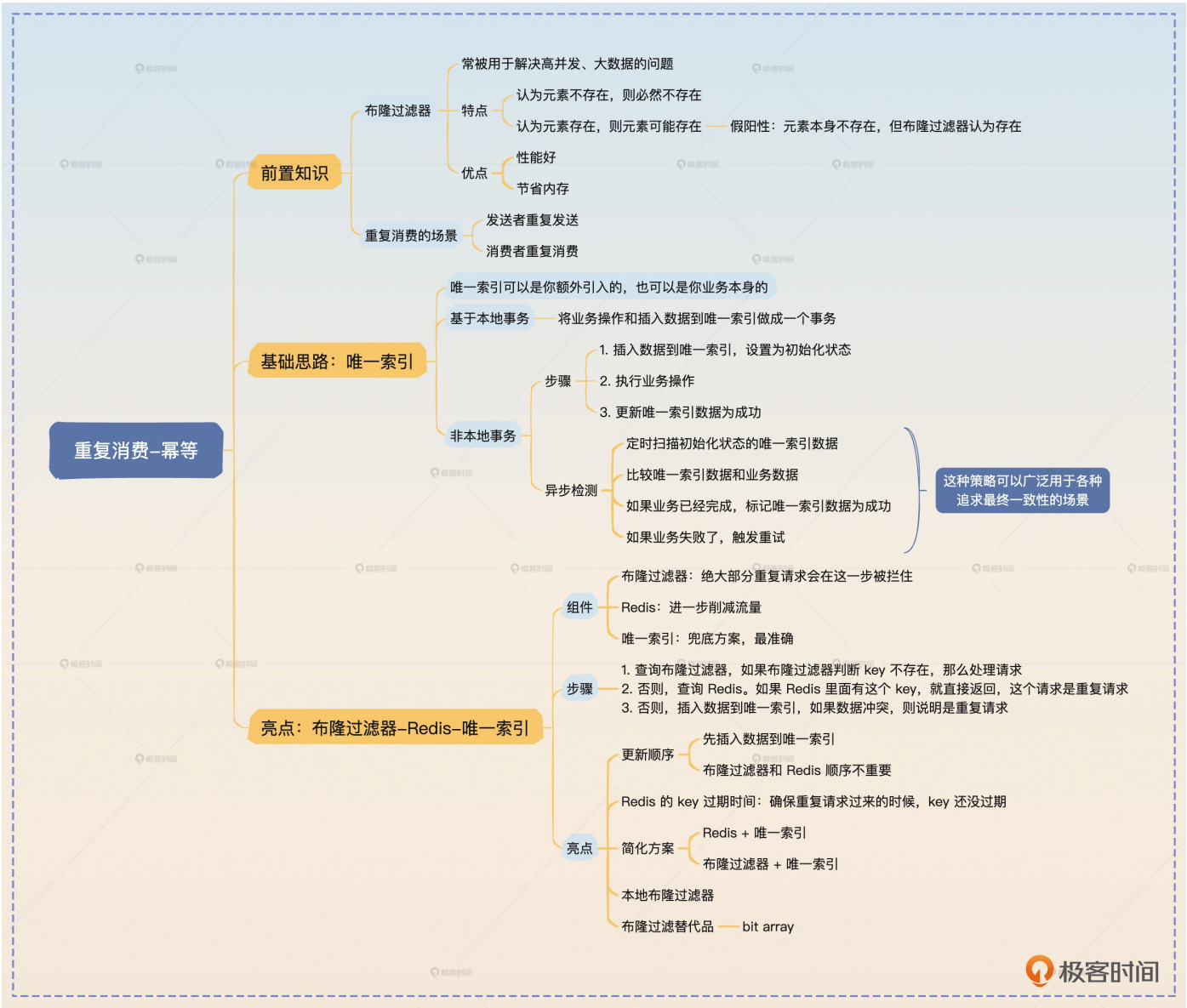
到这一步，幂等相关的内容你就已经刷到极致了。

面试思路总结

最后我们来总结一下这节课的内容。最开始我给你介绍了布隆过滤器，这是你理解后面方案的关键。然后我们分析了一下重复消费的可能原因，一个是生产者重复发送，另一个是消费者重复消费。

总的来说，解决重复消费的思路就一条：**让消费者做到幂等**。所以问题就是怎么做到幂等。那么最简单的方案就是**唯一索引**，然后我们深入分析了**有本地事务和没有本地事务这两种情况下怎么将数据插入到唯一索引**。

最后我给出了一个**布隆过滤器 + Redis + 唯一索引**的方案，这个方案是真正能撑住高并发的幂等方案。你要理解它的基本步骤，然后在面试过程中深入讨论如何更新系统、Redis 的 key 过期时间、简化方案、本地布隆过滤器、布隆过滤器的替代品这几个点，赢得面试的竞争优势。



思考题

最后请你来思考2个问题。

我在基本思路的唯一索引部分画了一张图，关键点是在 RR 隔离级别下重复请求的插入操作会被阻塞。那么如果隔离级别不是 RR 的话，你觉得会发生什么？

如果你的流量中，几乎不存在重复请求，比如说重复请求占比不到 1%，那么你觉得最后一个方案的效果如何？

欢迎你把你思考后的结果分享到评论区，也欢迎你把这节课分享给需要的朋友，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

上一篇 26 | 消息不丢失：生产者收到写入成功响应后消息一定不会丢失吗？

下一篇 28 | 架构设计：如果让你设计一个消息队列，你会怎么设计它的架构？

精选留言 12